Compiler Construction Lab - Assignment 8

Name: Dev Vachhani
PRN.: 24070122517
Batch: B3

Objective

To design and implement a program that reads an arithmetic expression containing binary operators (+, -, *, /), validates its syntax, and generates the corresponding syntax tree for a valid expression. If the expression is invalid, the program should display an appropriate error message.

Theory

A syntax tree represents the grammatical structure of an arithmetic expression. In this assignment, each operator is an internal node with two children, and each identifier or number is a leaf. Parentheses determine grouping during parsing but do not need to appear as nodes in the resulting tree.

The program uses two compiler tools:

- **Flex** generates the lexical analyzer. It reads the input and groups characters into tokens such as identifiers, numbers, operators (+, -, *, /), and parentheses.
- **Bison** generates the syntax analyzer. It checks whether the token sequence follows the arithmetic grammar and builds the tree when the expression is valid.

The grammar separates expressions, terms, and factors: expression →

expression + term

| expression - term

| term

term → term * factor

| term / factor

| factor

factor → (expression)

| identifier

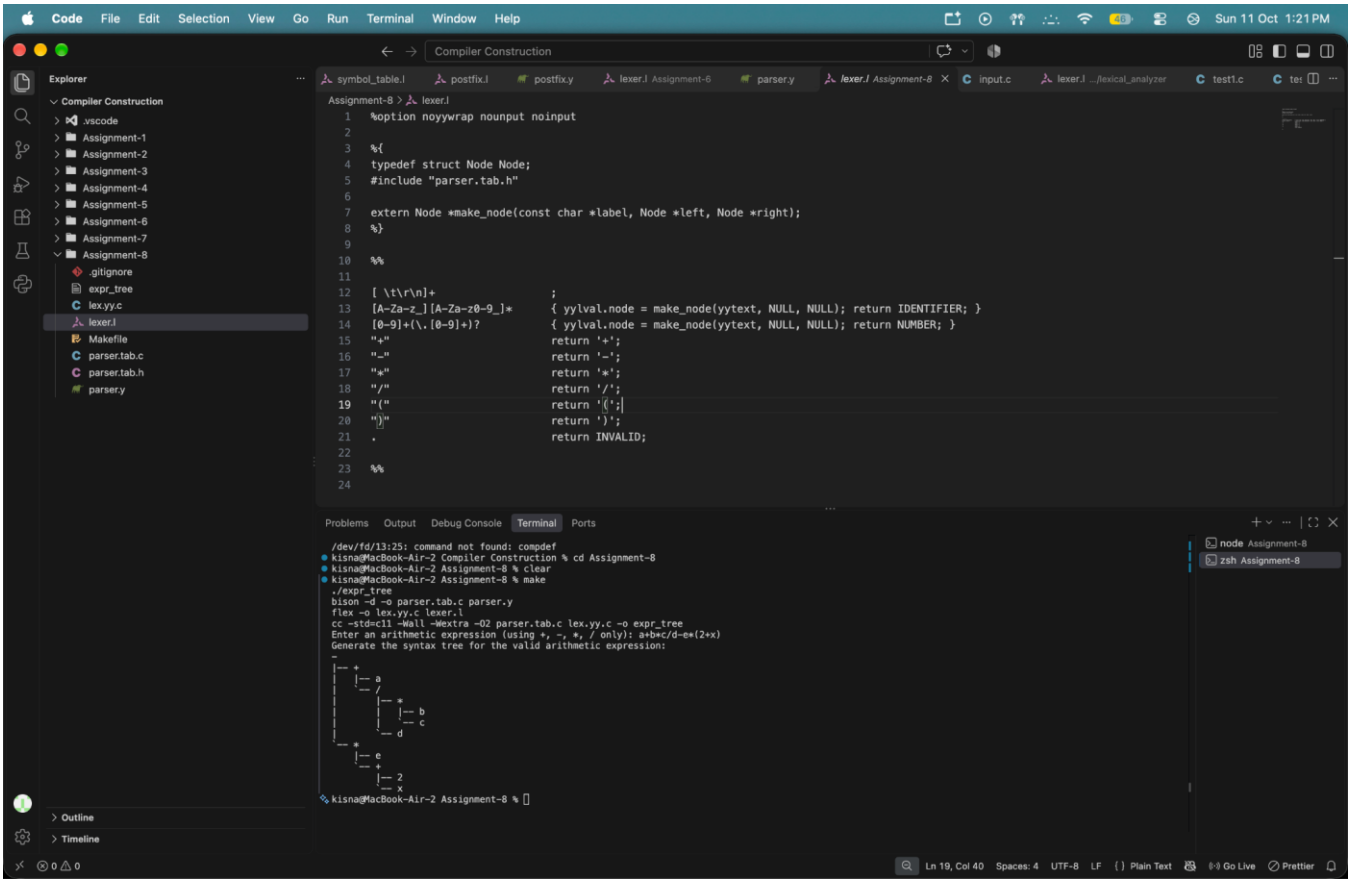
| number

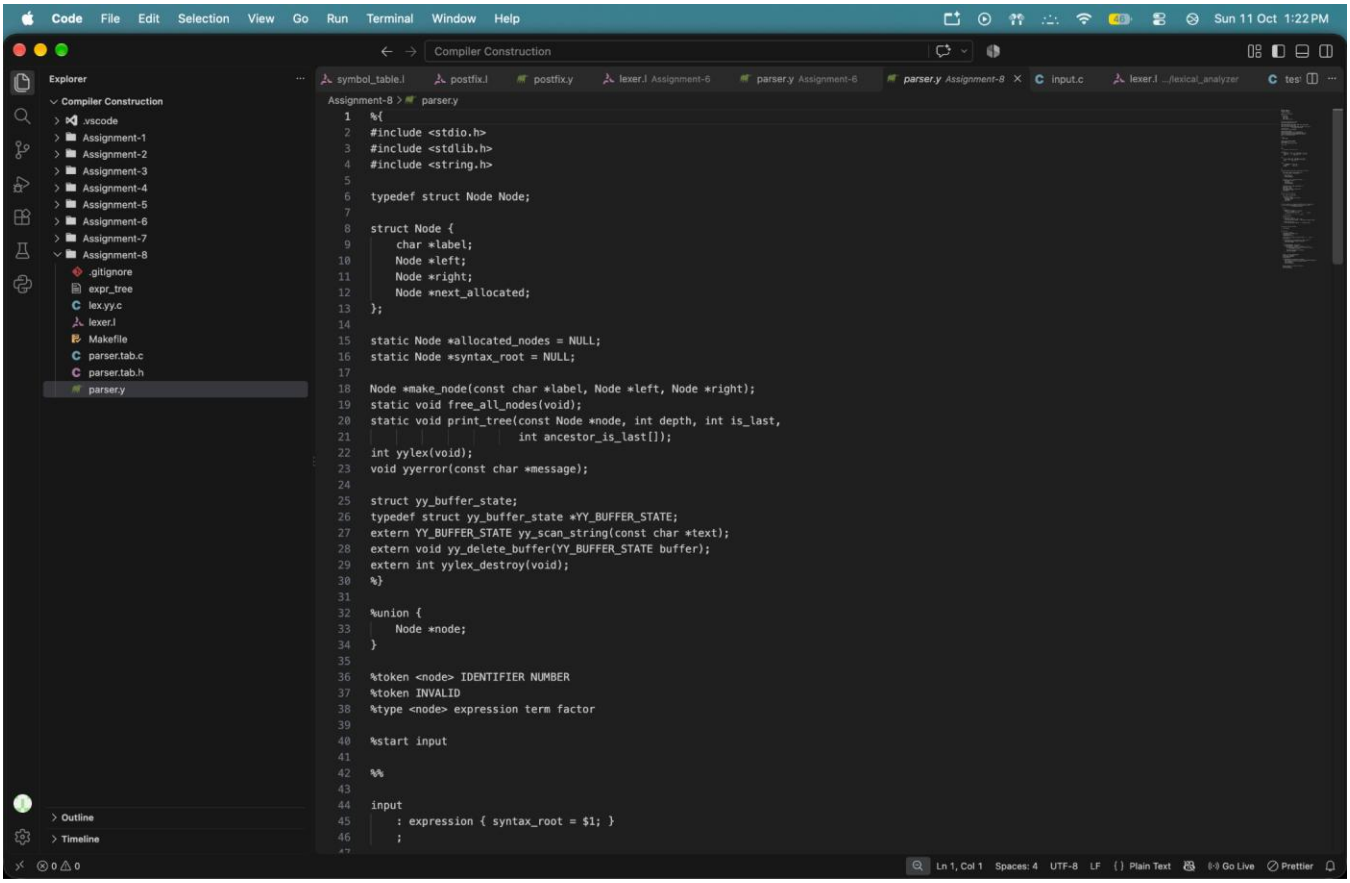
This structure gives multiplication and division higher precedence than addition and subtraction. The left-recursive rules make operators at the same precedence level evaluate from left to right. For example, a+b*c is grouped as a+(b*c).

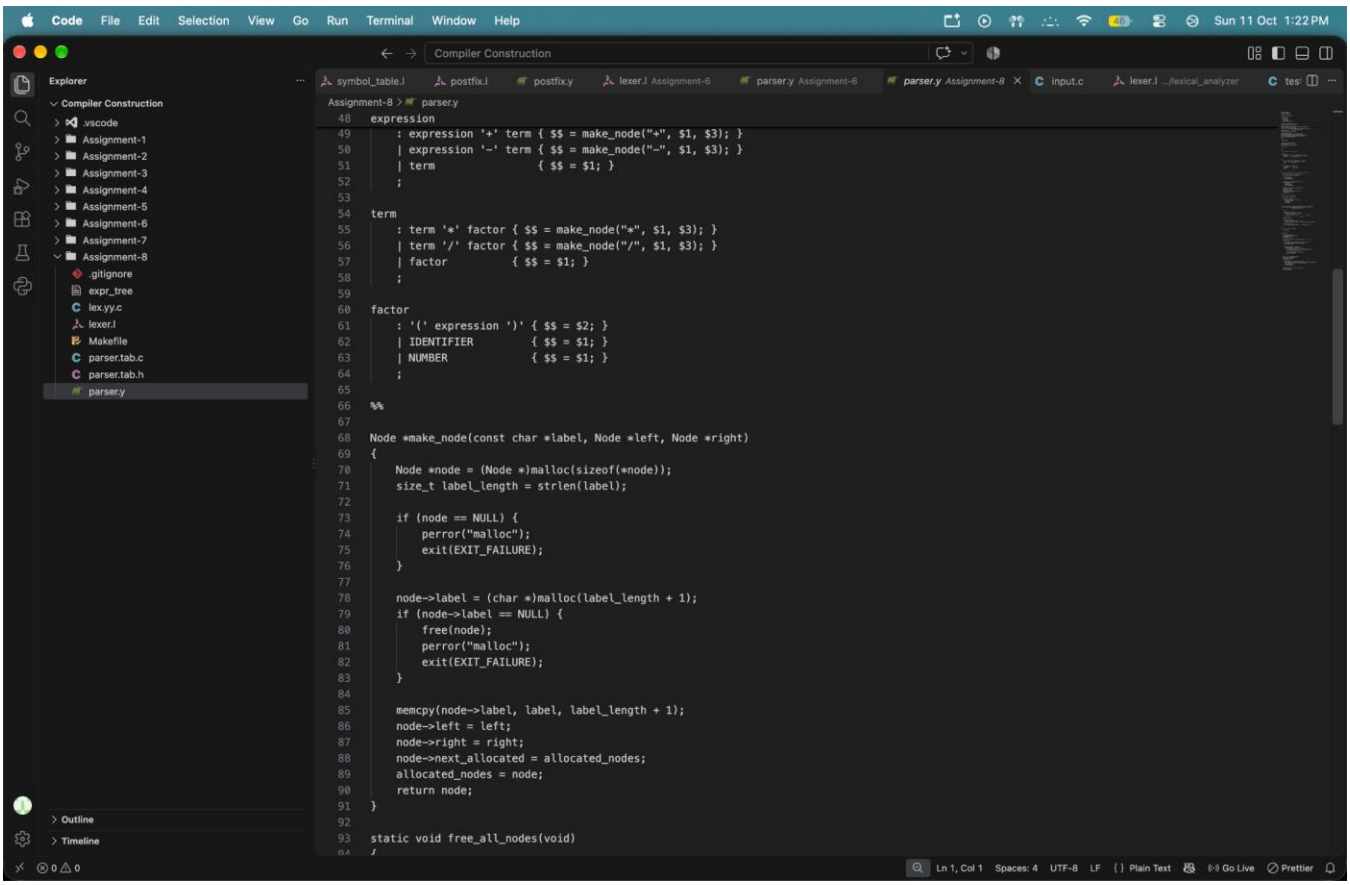
During parsing, the program creates a leaf node for each operand. When it recognizes a binary operation, it creates an operator node and attaches the left and right expression trees as its children. For example, a+b*c produces a + root, with a on the left and a * node on the right.

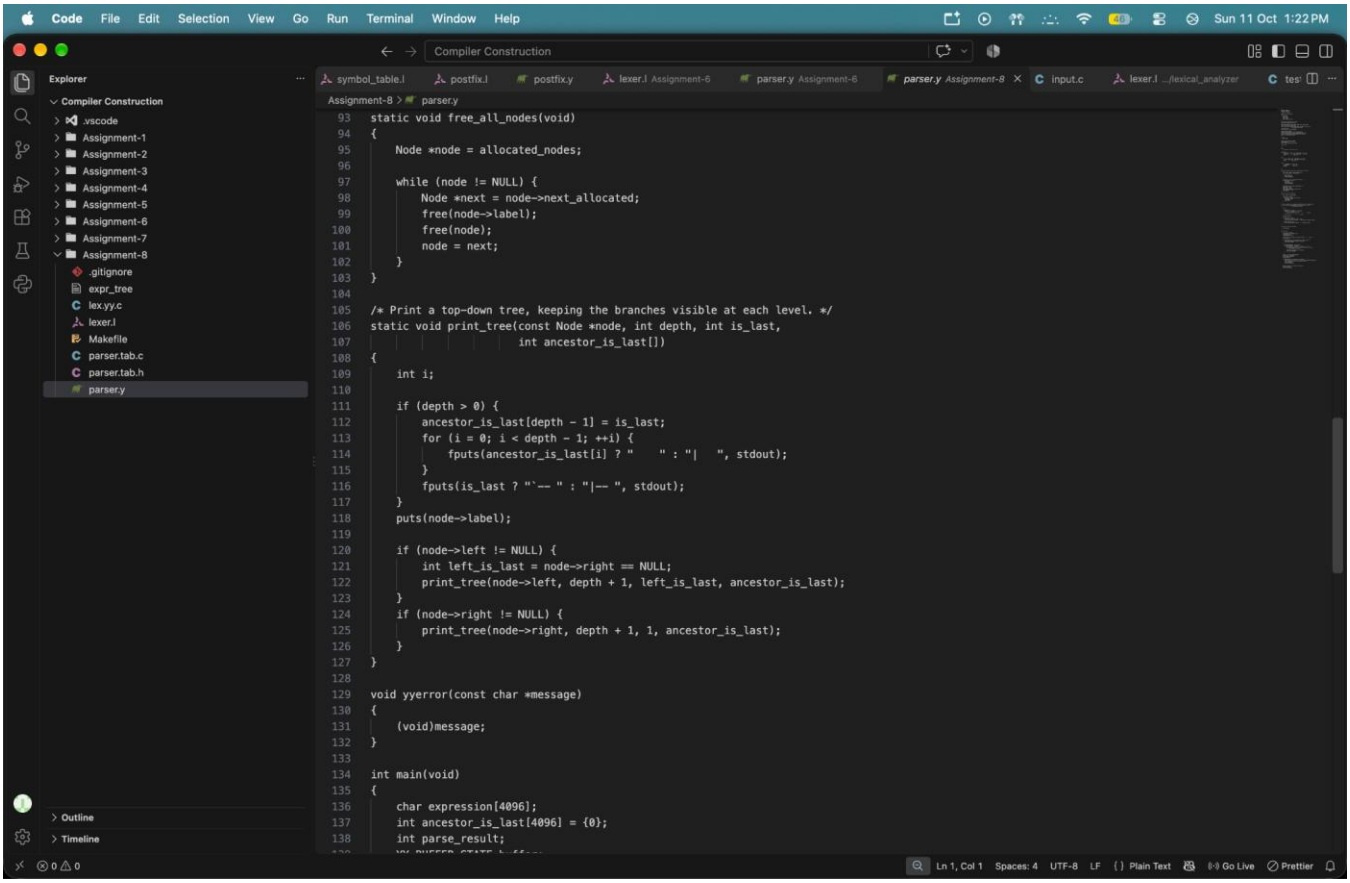
If the input does not follow the grammar—for example, if an operator is missing an operand or parentheses are unmatched—the parser reports that the arithmetic expression is invalid.

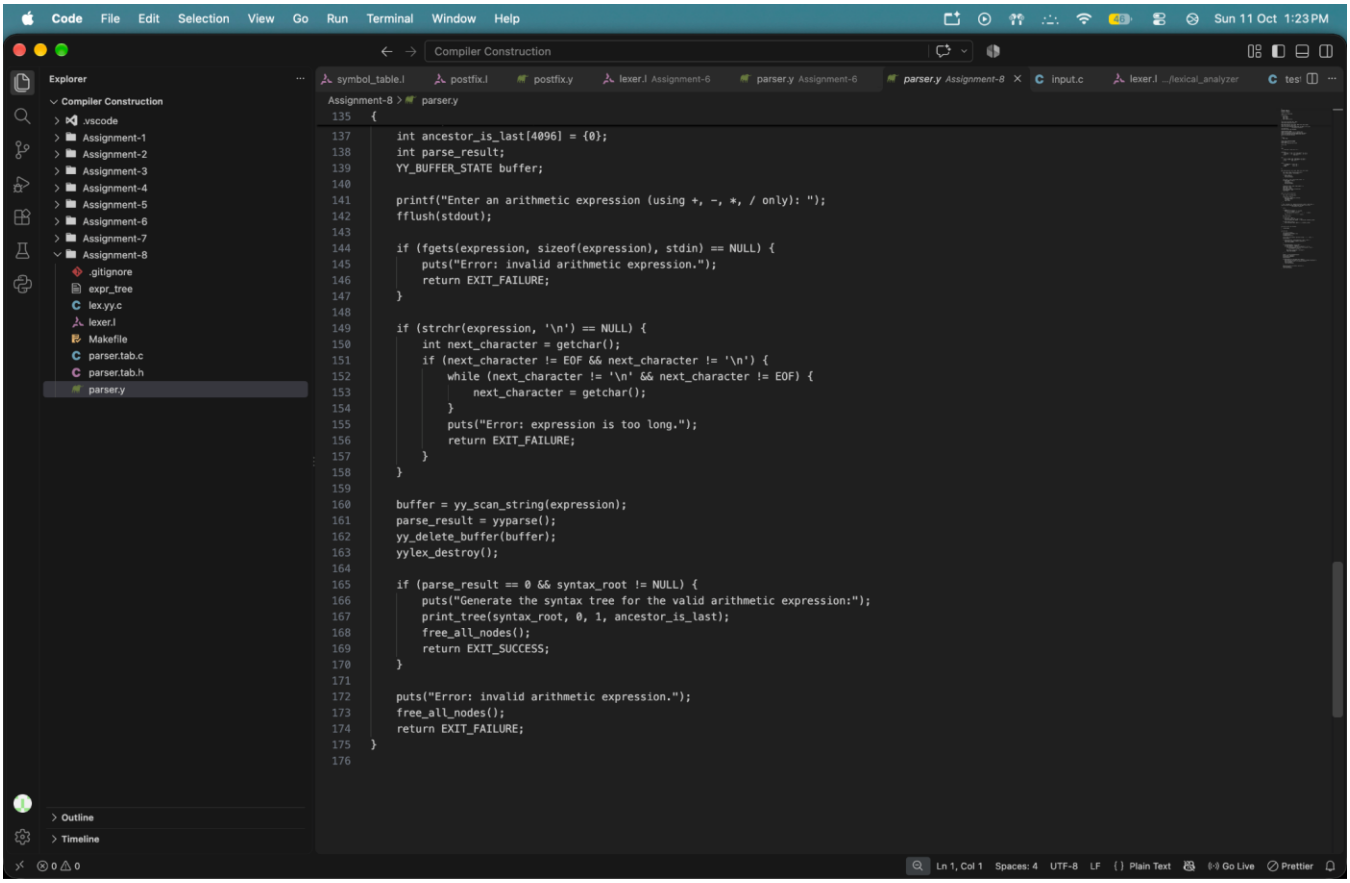
Screenshots and Execution Results




Code File Edit Selection View Go Run Terminal Window Help
Sun 11 Oct 1:22 PM
Compiler Construction
Explorer
Compiler Construction
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
.gitignore
expr_tree
lex.yy.c
lexer.l
Makefile
parser.tab.c
parser.tab.h
parser.y
Outline
Timeline
symbol_table.l
postfix.l
postfix.y
lexer.l Assignment-6
parser.y Assignment-6
parser.y Assignment-8
input.c
Assignment-8 > parser.y
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node Node;
struct Node {
char *label;
Node *left;
Node *right;
Node *next_allocated;
};
static Node *allocated_nodes = NULL;
static Node *syntax_root = NULL;
Node *make_node(const char *label, Node *left, Node *right);
static void free_all_nodes(void);
static void print_tree(const Node *node, int depth, int is_last,
int ancestor_is_last[]);
int yylex(void);
void yyerror(const char *message);
struct yy_buffer_state;
typedef struct yy_buffer_state *YY_BUFFER_STATE;
extern YY_BUFFER_STATE yy_scan_string(const char *text);
extern void yy_delete_buffer(YY_BUFFER_STATE buffer);
extern int yylex_destroy(void);
%}
%union {
Node *node;
}
%token <node> IDENTIFIER NUMBER
%token INVALID
%type <node> expression term factor
%start input
%%
input
: expression { syntax_root = $1; }
;
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Plain Text Go Live Prettier


Code File Edit Selection View Go Run Terminal Window Help
Sun 11 Oct 1:22 PM
Compiler Construction
Explorer
Compiler Construction
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
.gitignore
expr_tree
lex.yy.c
lexer.l
Makefile
parser.tab.c
parser.tab.h
parser.y
Outline
Timeline
symbol_table.l
postfix.l
postfix.y
lexer.l Assignment-6
parser.y Assignment-6
parser.y Assignment-8
input.c
Assignment-8 > parser.y
expression
: expression '+' term { $$ = make_node("+", $1, $3); }
| expression '-' term { $$ = make_node("-", $1, $3); }
| term { $$ = $1; }
;
term
: term '*' factor { $$ = make_node("*", $1, $3); }
| term '/' factor { $$ = make_node("/", $1, $3); }
| factor { $$ = $1; }
;
factor
: '(' expression ')' { $$ = $2; }
| IDENTIFIER { $$ = $1; }
| NUMBER { $$ = $1; }
;
%%
Node *make_node(const char *label, Node *left, Node *right)
{
Node *node = (Node *)malloc(sizeof(*node));
size_t label_length = strlen(label);
if (node == NULL) {
perror("malloc");
exit(EXIT_FAILURE);
}
node->label = (char *)malloc(label_length + 1);
if (node->label == NULL) {
free(node);
perror("malloc");
exit(EXIT_FAILURE);
}
memcpy(node->label, label, label_length + 1);
node->left = left;
node->right = right;
node->next_allocated = allocated_nodes;
allocated_nodes = node;
return node;
}
static void free_all_nodes(void)
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Plain Text Go Live Prettier


Code File Edit Selection View Go Run Terminal Window Help
Sun 11 Oct 1:22 PM
Compiler Construction
Explorer
Compiler Construction
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
.gitignore
expr_tree
lex.yy.c
lexer.l
Makefile
parser.tab.c
parser.tab.h
parser.y
symbol_table.l
postfix.l
postfix.y
lexer.l Assignment-6
parser.y Assignment-6
parser.y Assignment-8
input.c
Assignment-8 > parser.y
93 static void free_all_nodes(void)
94 {
95 Node *node = allocated_nodes;
96
97 while (node != NULL) {
98 Node *next = node->next_allocated;
99 free(node->label);
100 free(node);
101 node = next;
102 }
103 }
104
105 /* Print a top-down tree, keeping the branches visible at each level. */
106 static void print_tree(const Node *node, int depth, int is_last,
107 int ancestor_is_last[])
108 {
109 int i;
110
111 if (depth > 0) {
112 ancestor_is_last[depth - 1] = is_last;
113 for (i = 0; i < depth - 1; ++i) {
114 fputs(ancestor_is_last[i] ? " " : "| ", stdout);
115 }
116 fputs(is_last ? "`-- " : "|-- ", stdout);
117 }
118 puts(node->label);
119
120 if (node->left != NULL) {
121 int left_is_last = node->right == NULL;
122 print_tree(node->left, depth + 1, left_is_last, ancestor_is_last);
123 }
124 if (node->right != NULL) {
125 print_tree(node->right, depth + 1, 1, ancestor_is_last);
126 }
127 }
128
129 void yyerror(const char *message)
130 {
131 (void)message;
132 }
133
134 int main(void)
135 {
136 char expression[4096];
137 int ancestor_is_last[4096] = {0};
138 int parse_result;
Outline
Timeline
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Plain Text Go Live Prettier


Code File Edit Selection View Go Run Terminal Window Help
Sun 11 Oct 1:23 PM
Compiler Construction
Explorer
Compiler Construction
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
.gitignore
expr_tree
lex.yy.c
lexer.l
Makefile
parser.tab.c
parser.tab.h
parser.y
symbol_table.l
postfix.l
postfix.y
lexer.l Assignment-6
parser.y Assignment-6
parser.y Assignment-8
input.c
Assignment-8 > parser.y
135 {
137 int ancestor_is_last[4096] = {0};
138 int parse_result;
139 YY_BUFFER_STATE buffer;
140
141 printf("Enter an arithmetic expression (using +, -, *, / only): ");
142 fflush(stdout);
143
144 if (fgets(expression, sizeof(expression), stdin) == NULL) {
145 puts("Error: invalid arithmetic expression.");
146 return EXIT_FAILURE;
147 }
148
149 if (strchr(expression, '\n') == NULL) {
150 int next_character = getchar();
151 if (next_character != EOF && next_character != '\n') {
152 while (next_character != '\n' && next_character != EOF) {
153 next_character = getchar();
154 }
155 puts("Error: expression is too long.");
156 return EXIT_FAILURE;
157 }
158 }
159
160 buffer = yy_scan_string(expression);
161 parse_result = yyparse();
162 yy_delete_buffer(buffer);
163 yylex_destroy();
164
165 if (parse_result == 0 && syntax_root != NULL) {
166 puts("Generate the syntax tree for the valid arithmetic expression:");
167 print_tree(syntax_root, 0, 1, ancestor_is_last);
168 free_all_nodes();
169 return EXIT_SUCCESS;
170 }
171
172 puts("Error: invalid arithmetic expression.");
173 free_all_nodes();
174 return EXIT_FAILURE;
175 }
176
Outline
Timeline
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Plain Text Go Live Prettier